
*THÉORIE DES GRAPHS :
RECHERCHE DES CHEMINS LES
PLUS COURTS PAR
L'ALGORITHME DE FLOYD-
WARSHALL*

PROFESSEUR : MELEKHOVA OLGA

Sommaire

<i>Introduction</i>	<i>3</i>
<i>L'algorithme de Floyd-Warshall</i>	<i>3</i>
<i>Implémentation.....</i>	<i>3</i>
<i>Exemple d'application réelle.....</i>	<i>3</i>
<i>Conclusion</i>	<i>6</i>

Introduction

L'objectif de ce projet est d'implémenter l'algorithme de Floyd-Warshall, qui permet de calculer les chemins de valeur minimale entre les sommets d'un graphe valué qu'il soit orienté ou non. Cet algorithme constitue une extension de l'algorithme de Warshall, initialement dédié au calcul de la fermeture transitive, mais qui permet cette fois-ci la conservation des chemins de coût minimal. Le programme développé prend en charge la lecture d'un graphe depuis un fichier texte, son stockage en mémoire, l'exécution de l'algorithme avec affichage des étapes intermédiaires, la détection des circuits absorbants, et surtout, l'affiche des chemins minimaux à l'utilisateur.

L'algorithme de Floyd-Warshall

L'algorithme de Floyd-Warshall utilise deux matrices de taille $n \times n$, notées *dist* et *suiv*. La matrice *dist* contient les distances minimales connues entre chaque paire de sommets, et la matrice *suiv* mémorise le prochain sommet à emprunter sur le chemin optimal, permettant de reconstruire les trajets.

L'initialisation consiste à poser $\text{dist}[i][j]$ = valeur de l'arc (i,j) s'il existe, $\text{dist}[i][i] = 0$, et $\text{dist}[i][j] = \text{INF}$ sinon. La matrice *suiv* est initialisée avec $\text{suiv}[i][j] = j$ lorsqu'un arc direct $(i \rightarrow j)$ existe, et « None » sinon.

L'algorithme effectue ensuite n itérations (une par sommet intermédiaire k). À chaque étape, pour tout couple (i, j) , on vérifie si passer par le sommet k améliore le chemin connu : si $\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$, alors on met à jour $\text{dist}[i][j]$ et $\text{suiv}[i][j] \leftarrow \text{suiv}[i][k]$.

À la fin des n itérations, si un $\text{dist}[i][i]$ est strictement négatif alors le graphe contient un circuit absorbant : un cycle dont la somme des valeurs est négative, rendant impossible le calcul d'un chemin minimal.

Implémentation

Le programme est développé en Python, sous forme de fonctions documentées correspondant à chaque étape du traitement.

Lecture et stockage : la fonction `lire_graphe(numero)` lit le fichier `graphes/graphe{numero}.txt` et retourne le nombre de sommets ainsi qu'une liste de triplets (source, destination, valeur). Une fois chargé, le graphe n'est plus accédé depuis le fichier : toutes les opérations reposent uniquement sur les structures en mémoire.

Structure de données : le graphe est représenté par une matrice d'adjacence *dist0* (liste de listes Python), construite par `construire_matrice(n, arcs)`. Les matrices *dist* et *suiv* sont copiées à chaque itération pour conserver l'historique complet des étapes.

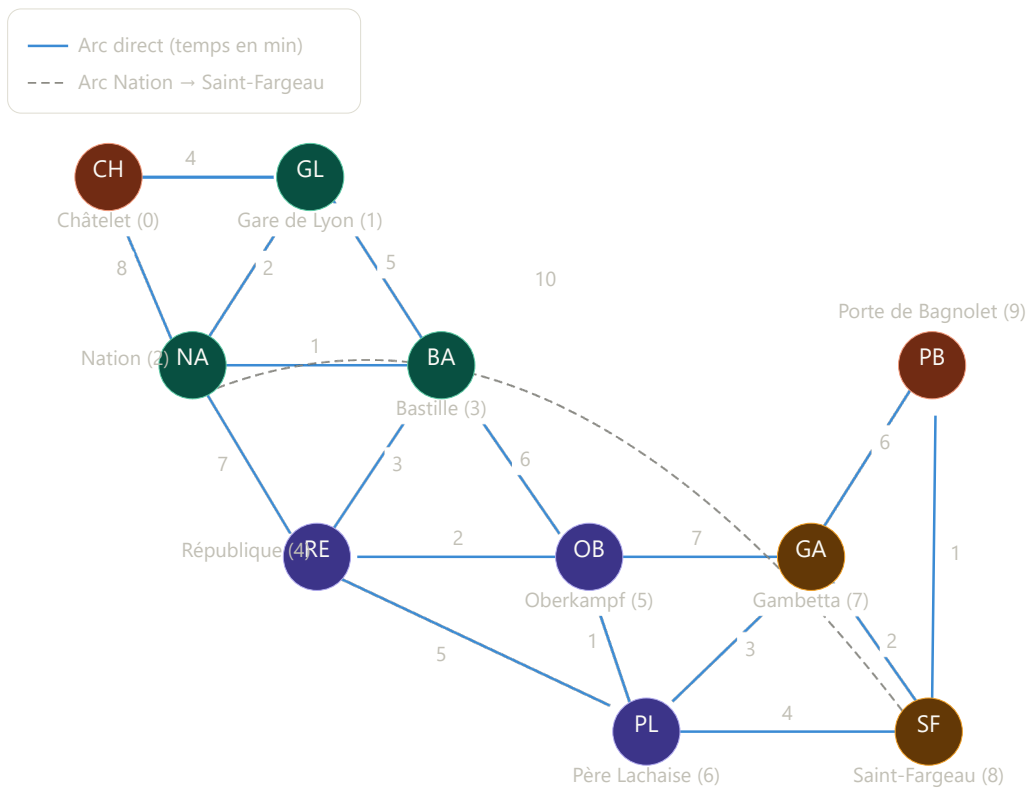
Format des fichiers :

- ligne 1 $\rightarrow$ nombre de sommets
- ligne 2 $\rightarrow$ nombre d'arcs, puis une ligne par arc sous la forme source destination valeur.

Exemple d'application réelle

Dans le cadre de ce projet, nous avons modélisé un sous-réseau du métro parisien afin d'illustrer concrètement l'utilité de l'algorithme de Floyd-Warshall. L'objectif est de déterminer, pour chaque paire de stations, le temps de trajet minimal, en tenant compte des correspondances possibles entre stations intermédiaires.

Le réseau est représenté par un graphe orienté et valué de 10 sommets et 18 arcs. Chaque sommet correspond à une station, et chaque arc représente une liaison directe dont la valeur est le temps de trajet en minutes.



Sommet	Station	Initiales
0	Châtelet	CH
1	Gare de Lyon	GL
2	Nation	NA
3	Bastille	BA
4	République	RE
5	Oberkampf	OB
6	Père Lachaise	PL
7	Gambetta	GA
8	Saint-Fargeau	SF
9	Porte de Bagnolet	PB

Avant l'exécution de l'algorithme, on construit les matrices d'adjacence dist_0 et suiv_0 directement à partir des arcs du graphe. dist_0 ne contient que les temps de trajet des liaisons directes entre stations, toutes les autres cases étant à $+\infty$. suiv_0 indique uniquement le prochain sommet à emprunter pour les voisins immédiats de chaque station.

[dist0]											
	0	1	2	3	4	5	6	7	8	9	
0	0	4	8	INF	INF	INF	INF	INF	INF	INF	
1	INF	0	2	5	INF	INF	INF	INF	INF	INF	
2	INF	INF	0	1	7	INF	INF	INF	10	INF	
3	INF	INF	INF	0	3	6	INF	INF	INF	INF	
4	INF	INF	INF	INF	0	2	5	INF	INF	INF	
5	INF	INF	INF	INF	INF	0	1	7	INF	INF	
6	INF	INF	INF	INF	INF	INF	0	3	4	INF	
7	INF	INF	INF	INF	INF	INF	INF	0	2	6	
8	INF	INF	INF	INF	INF	INF	INF	INF	0	1	
9	INF	INF	INF	INF	INF	INF	INF	INF	INF	0	

[suiv0]											
	0	1	2	3	4	5	6	7	8	9	
0	-	1	2	-	-	-	-	-	-	-	
1	-	-	2	3	-	-	-	-	-	-	
2	-	-	-	3	4	-	-	-	8	-	
3	-	-	-	-	4	5	-	-	-	-	
4	-	-	-	-	-	5	6	-	-	-	
5	-	-	-	-	-	-	6	7	-	-	
6	-	-	-	-	-	-	-	7	8	-	
7	-	-	-	-	-	-	-	-	8	9	
8	-	-	-	-	-	-	-	-	-	9	
9	-	-	-	-	-	-	-	-	-	-	

Après exécution de l'algorithme de Floyd-Warshall, aucun circuit absorbant n'est détecté. Les matrices finales dist10 et suiv10 sont présentées ci-dessous.

[k=9 dist10]											
	0	1	2	3	4	5	6	7	8	9	
0	0	4	6	7	10	12	13	16	16	17	
1	INF	0	2	3	6	8	9	12	12	13	
2	INF	INF	0	1	4	6	7	10	10	11	
3	INF	INF	INF	0	3	5	6	9	10	11	
4	INF	INF	INF	INF	0	2	3	6	7	8	
5	INF	INF	INF	INF	INF	0	1	4	5	6	
6	INF	INF	INF	INF	INF	INF	0	3	4	5	
7	INF	INF	INF	INF	INF	INF	INF	0	2	3	
8	INF	INF	INF	INF	INF	INF	INF	INF	0	1	
9	INF	INF	INF	INF	INF	INF	INF	INF	INF	0	

[k=9 suiv10]

	0	1	2	3	4	5	6	7	8	9
0	-	1	1	1	1	1	1	1	1	1
1	-	-	2	2	2	2	2	2	2	2
2	-	-	-	3	3	3	3	3	8	8
3	-	-	-	-	4	4	4	4	4	4
4	-	-	-	-	-	5	5	5	5	5
5	-	-	-	-	-	-	6	6	6	6
6	-	-	-	-	-	-	-	7	8	8
7	-	-	-	-	-	-	-	-	8	8
8	-	-	-	-	-	-	-	-	-	9
9	-	-	-	-	-	-	-	-	-	-

[OK] Aucun circuit absorbant.

La matrice dist10 permet de lire directement le temps de trajet minimal entre deux stations quelconques. Par exemple :

- Châtelet → Porte de Bagnolet : 17 minutes, en passant par Gare de Lyon → Nation → Saint-Fargeau → Porte de Bagnolet
- Châtelet → Oberkampf : 12 minutes.
- Nation → Porte de Bagnolet : 11 minutes.
- Bastille → Père Lachaise : 6 minutes, via République → Oberkampf → Père Lachaise.

Les cases à INF indiquent qu'il n'existe aucun chemin dans ce sens, ce qui est cohérent avec la nature orientée du réseau, on ne peut pas remonter le réseau dans ce sous-graphe.

La matrice suiv10 permet quant à elle de reconstruire le chemin exact à emprunter. Pour aller de la station 0 à la station 9 par exemple, la case (0,9) indique 1 : on se dirige d'abord vers Gare de Lyon, puis en suivant la chaîne de la matrice on obtient le chemin complet jusqu'à Porte de Bagnolet.

Cet exemple montre que l'algorithme de Floyd-Warshall est particulièrement adapté aux réseaux de transport : en une seule exécution, il fournit l'ensemble des trajets optimaux pour tous les couples de stations, ce qui serait impossible à obtenir manuellement sur un réseau de grande taille.

Conclusion

Ce projet nous a permis d'implémenter de façon complète l'algorithme de Floyd-Warshall, depuis la lecture des données jusqu'à la restitution des chemins minimaux. La principale difficulté rencontrée a été la bonne gestion des valeurs infinies lors des calculs intermédiaires, ainsi que la reconstruction correcte des chemins via la matrice suiv. L'exemple d'application réel du sous-réseau de métro parisien, modélisant 10 stations de Châtelet à Porte de Bagnolet, illustre concrètement l'utilité de cet algorithme dans des problèmes réels d'optimisation de trajets. Une extension possible serait d'adapter le programme à des graphes non orientés, d'enrichir le réseau avec davantage de stations, ou de directement intégrer une visualisation graphique du réseau.